Verification Lab 1

Mohammad Attari & William Marnfeldt

1 Introduction

The purpose of this lab is to help you break the ice with SystemVerilog and introduce you to some of its useful constructs and features. By following the lab you will learn the concepts below.

- Randomization and constrained randomization.
- Working with threads using `fork-join` statements[1].
- Assertions and properties.
- Coverage, coverpoints, and cover bins.

2 The file structure

The lab is divided into 5 parts and organized as follows:

```
Lab 1
├── lab_01.sv
├── lab_02.sv
├── lab_03.sv
├── lab_04.sv
└── lab_05.sv
```

Each lab is devoted to one of the topics listed above. The code for each part is written in a manner that is self-explanatory, so read the code and perform the simulation, and try to see if you can figure out what makes it tick, and what makes it kick. If there's something you're not sure about, you can Google it out[2] or consult your favorite AI friend(s). Additionally, for a rather comprehensive coverage of SystemVerilog (both design and verification) you can check out the book "Introduction to SystemVerilog", which is accessible (PDF and EPUB files) from the Springer library through this link[3] Heres a few other nice resources as well:

https://link.springer.com/book/10.1007/978-3-030-71319-5

https://www.chipverify.com/ – Good Lookup for syntax
https://verificationacademy.com/ – Any forum post from Dave59 is gold.
https://www.doulos.com/knowhow/systemverilog/ – A treasure trove.

If all your attempts prove futile, ask your omniscient TA:s for help. When you have a solid grasp of what is going on, then move on and carry out the tasks listed in the code, which are also reproduced here for convenience.

[1] No needles were hurt during the making of this lab.
[2] Bing it out, anyone?
[3] Needs Lund University IP address

3 Tasks

3.1 Lab 01 - Scope Randomization

Task 1:
Create a function that randomizes `my_state` 16 times in a way that it excludes `START` from the randomization.

Task 2:
Create a function that randomizes data 16 times in a way that it is twice as likely to get `data <= 10` than it is to get `data >= 200` (and 0 likelihood for `10 < data < 200`).

Task 3:
Create a function that randomizes data 16 times in a way that if `my_state` is `[S1:S6]` then `50 <= data <= 60` or `100 <= data <= 150`, otherwise `data < 20`.

3.2 Lab 02 - Class Randomization

Task 1:
Uncomment the line in the `Packet` class containing the `payload_size_conflict` constraint, and see what transpires. What on earth is happening?
when ever payload size is 5, it throws an error message for task 2

Task 2:
Create a class named `Task2` and in its `run` task call `randomize` in a way to only apply randomization to `src` and `payload` members of `Packet`.
dest:x->0

Task 3:
Create a class named `MyBus` that extends `Bus` and change its `address_rule` so that `addr[7:0]` can not be 8'hff and also not in the range 8'h11:8'h77
Randomize and test `MyBus` in a new class named `Task3`, but turn off `data_rule1`.

3.3 Lab 03 - Threading

Task 1:
Create a new class named `Task1` that is a carbon copy of the `Test1` class with the exception that the `fork-join` block only creates 1 thread.

Task 2:
Create a new class named `Task2` that is a carbon copy of the `Test5` class with the exception that the threads created in the `fork-join` blocks need to finish first before moving on.

3.4 Lab 04 - Assertions

Task 1:
Add an assertion that checks if `b`, `c`, and `d` are high this cycle, then `d` must be high 2 cycles later.

Task 2:
Same as `Task 1` except that reset disables the check. make reset = 0

Task 3:
Add an assertion that checks `data <= 200` at positive clock edges.

Task 4:
Add an assertion that checks if `a` is high this cycle, and `c` is high the cycle after, and `b` is high 2 cycles after `a` was high, then `d` must be high 3 cycles after `a` was high.
So for example if `a` is high at cycle 1 and `c` is high at cycle 2 and `b` is high at cycle 3 then `d` must be high at cycle 4.

3.5 Lab 05 - Coverage

You can check the coverage from `View->Coverage->Covergroups` in QuestaSim.

Task 1:
Change the testbench to achieve 100% coverage for `covergroup_1`. initial cov 82.81%
run for 3000ns

Task 2:
Change `covergroup_2`'s `data` coverpoint so that it only creates 1 bin for 0, 1, 2, 5, 100 instead of 5 individual bins. remove []

Task 3:
Change `covergroup_3` and achieve the same functionality without using the `default` keyword.

Task 4:
Change the testbench to achieve 100% cross coverage for `covergroup_5`.